TRAFFIC LIGHT CONTROLLER

BY: AVERY MARTINEZ AND CARLOS MEDINA

GROUP OBJECTIVE

- Design a traffic light controller using STM32
- Implement multitasking with FreeRTOS
- Handle pedestrian requests using buttons
- Use queues for task communication
- Use a mutex for shared variables protection

HARDWARE COMPONENTS

- STM32 Microcontroller
- LEDs (Red, Yellow, Green)
- Push Buttons (2 pedestrian inputs)
- Timer (TIM2 for timing)

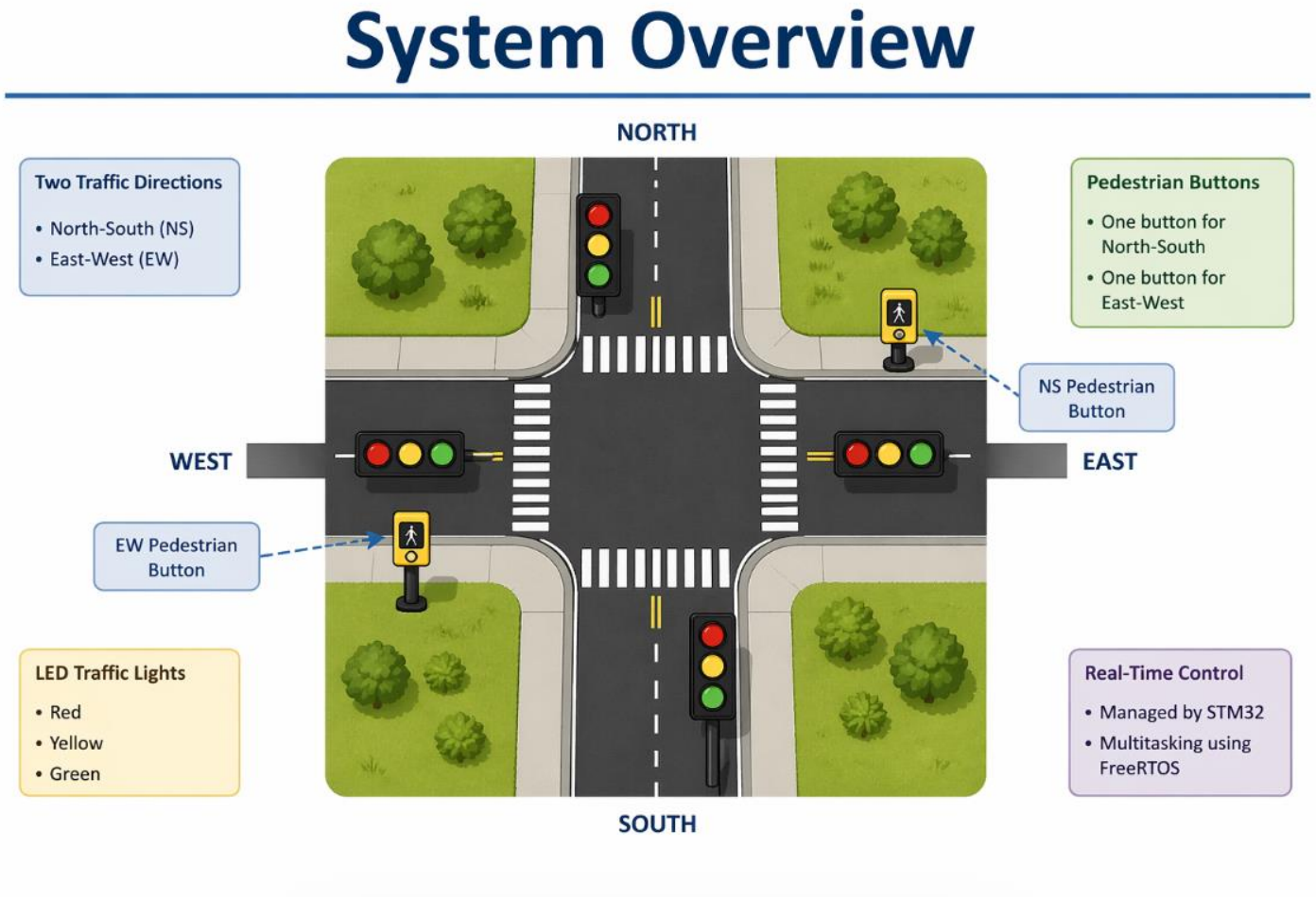
SYSTEM OVERVIEW

- Two traffic directions:

North-South (NS)

East-West (EW)

- Pedestrian buttons for each direction
- LED-based traffic lights
- Real-time control using tasks



```
#define YELLOW_TIME_MS          4000
#define ALL_RED_TIME_MS         5000

/* North traffic light */
#define N_RED_PORT          GPIOC
#define N_RED_PIN           GPIO_PIN_7
#define N_YELLOW_PORT       GPIOA
#define N_YELLOW_PIN        GPIO_PIN_9
#define N_GREEN_PORT        GPIOA
#define N_GREEN_PIN         GPIO_PIN_8

/* South traffic light */
#define S_RED_PORT          GPIOB
#define S_RED_PIN           GPIO_PIN_5
#define S_YELLOW_PORT       GPIOB
#define S_YELLOW_PIN        GPIO_PIN_6
#define S_GREEN_PORT        GPIOA
#define S_GREEN_PIN         GPIO_PIN_10

/* East traffic light */
#define E_RED_PORT          GPIOA
#define E_RED_PIN           GPIO_PIN_0
#define E_YELLOW_PORT       GPIOA
#define E_YELLOW_PIN        GPIO_PIN_1
#define E_GREEN_PORT        GPIOA
#define E_GREEN_PIN         GPIO_PIN_4

/* West traffic light */
#define W_RED_PORT          GPIOB
#define W_RED_PIN           GPIO_PIN_0
#define W_YELLOW_PORT       GPIOC
#define W_YELLOW_PIN        GPIO_PIN_1
#define W_GREEN_PORT        GPIOC
#define W_GREEN_PIN         GPIO_PIN_0

#define NE_BLUE_PORT        GPIOB
#define NE_BLUE_PIN         GPIO_PIN_10
#define SW_BLUE_PORT        GPIOB
#define SW_BLUE_PIN         GPIO_PIN_4

#define NS_BUTTON_PORT      GPIOA
#define NS_BUTTON_PIN       GPIO_PIN_6
#define EW_BUTTON_PORT      GPIOA
#define EW_BUTTON_PIN       GPIO_PIN_7
```

Figure 5: Pin & Port definitions

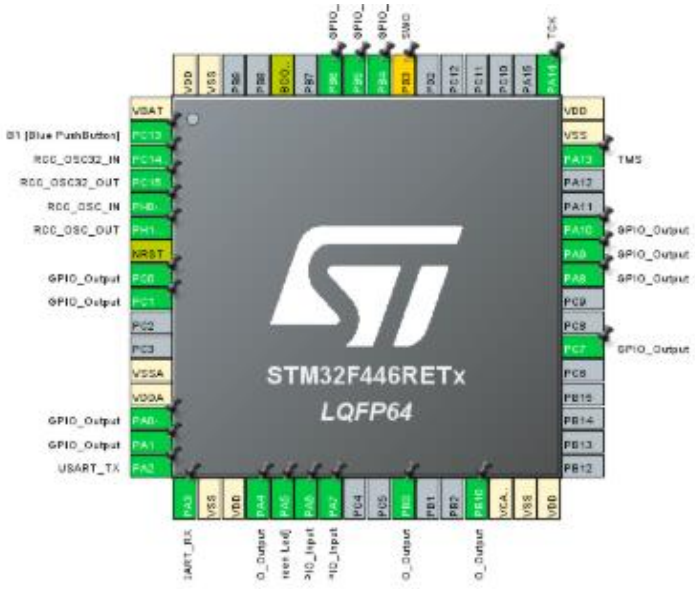


Figure 1: Pinout Configuration

SOFTWARE ARCHITECTURE

Pin & Port Configuration

- GPIO pins assigned to LEDs and buttons
- Covers all directions (NS & EW)
- Left: code definitions
- Right: STM32 pinout

RTOS TASKS

- **TrafficTask** → controls state transitions
- **PedestrianTask** → reads button inputs
- **BlinkTask** → blinks request indicator LEDs

Each task has its own priority and stack size

```
osThreadDef(TrafficTask, Traffic_Task, osPriorityNormal, 0, 256);
TrafficTaskHandle = osThreadCreate(osThread(TrafficTask), NULL);

// Pedestrian Button Task
osThreadDef(PedTask, PedestrianTask, osPriorityBelowNormal, 0, 128);
osThreadCreate(osThread(PedTask), NULL);

// LED Blink Task
osThreadDef(BlinkT, BlinkTask, osPriorityLow, 0, 128);
osThreadCreate(osThread(BlinkT), NULL);
/* USER CODE END RTOS_THREADS */
```

QUEUE IMPLEMENTATION

```c
/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
trafficQueue = xQueueCreate(10, sizeof(TrafficEvent));
/* USER CODE END RTOS_QUEUES */
```

- Queue created for timer events
- Used to send events from TIM2 interrupt to TrafficTask
- Improves synchronization
- Avoids constant polling

QUEUE FLOW

- TIM2 interrupt generates an event
- Event is sent to the queue
- TrafficTask receives the event
- State machine updates the traffic lights

TIMER USAGE

```
StartTrafficTimer(void)

stateElapsedMs = 0;
__HAL_TIM_SET_COUNTER(&h
HAL_TIM_Base_Start_IT(&h
```

- TIM2 configured as 1 ms time base
- Increments elapsed state time
- Controls traffic light timing
- Sends tick event through queue

TRAFFIC TIMING

- Minimum Green Time = 2000 ms
- Green Time = 15000 ms
- Yellow Time = 4000 ms
- All Red Time = 5000 ms

```c
/* USER CODE BEGIN PD */
/* Traffic timing */
#define MIN_GREEN_TIME_MS        2000
#define GREEN_TIME_MS            15000
#define YELLOW_TIME_MS           4000
#define ALL_RED_TIME_MS          5000
```

TRAFFIC LIGHT STATE MACHINE

- System controlled using a state machine
- States:

NS Green → NS Yellow → All Red

EW Green → EW Yellow → All Red

- Transitions based on timer and requests
- Pedestrian requests can reduce green time
- Ensures safe and continuous traffic flow

```c
void ProcessState(void)
{
    stateElapsedMs = 0;

    switch(currentState)
    {
        case STATE_NS_GREEN:
            SetNSGreen();
            break;

        case STATE_NS_YELLOW:
            SetNSYellow();
            break;

        case STATE_ALL_RED_TO_EW:
            AllLightsRed();
            break;

        case STATE_EW_GREEN:
            SetEWGreen();
            break;

        case STATE_EW_YELLOW:
            SetEWYellow();
            break;

        case STATE_ALL_RED_TO_NS:
            AllLightsRed();
            break;

        default:
            currentState = STATE_NS_GREEN;
            SetNSGreen();
            break;
    }
}
```

```c
for(;;)
{
  if (xQueueReceive(trafficQueue, &event, portMAX_DELAY) == pdPASS)
  {
    switch(currentState)
    {
      case STATE_NS_GREEN:
        /* Logic: If East-West is waiting (ew_ped_request),
           transition to Yellow after 2 seconds.
        */
        uint32_t ns_limit = (ew_ped_request == 1) ? MIN_GREEN_TIME_MS : GREEN_TIME_MS;

        if (stateElapsedMs >= ns_limit)
        {
          currentState = STATE_NS_YELLOW;
          ProcessState();
        }
        break;

      case STATE_NS_YELLOW:
        if (stateElapsedMs >= YELLOW_TIME_MS)
        {
          ns_ped_request = 0; // Clear NS request as lights turn Red
          currentState = STATE_ALL_RED_TO_EW;
          ProcessState();
        }
        break;

      case STATE_ALL_RED_TO_EW:
        if (stateElapsedMs >= ALL_RED_TIME_MS)
        {
          currentState = STATE_EW_GREEN;
          ProcessState();
        }
        break;

      case STATE_EW_GREEN:
        /* Logic: If North-South is waiting (ns_ped_request),
           transition to Yellow after 2 seconds.
        */
        uint32_t ew_limit = (ns_ped_request == 1) ? MIN_GREEN_TIME_MS : GREEN_TIME_MS;

        if (stateElapsedMs >= ew_limit)
        {
          currentState = STATE_EW_YELLOW;
          ProcessState();
        }
        break;

      case STATE_EW_YELLOW:
        if (stateElapsedMs >= YELLOW_TIME_MS)
        {
          ew_ped_request = 0; // Clear EW request as lights turn Red
          currentState = STATE_ALL_RED_TO_NS;
          ProcessState();
        }
        break;

      case STATE_ALL_RED_TO_NS:
        if (stateElapsedMs >= ALL_RED_TIME_MS)
        {
          currentState = STATE_NS_GREEN;
          ProcessState();
        }
        break;

      default:
        currentState = STATE_NS_GREEN;
```

TRAFFICTASK

- Controls traffic light states
- Implements state machine
- Receives timer events from queue
- Decides when to change states

PEDESTRIANTASK

- Reads button inputs
- Detects pedestrian requests
- Sets request flags:

ns_ped_request

ew_ped_request

```c
void PedestrianTask(void const * argument)
{
  for(;;)
  {
    if(HAL_GPIO_ReadPin(GPIOA, NS_BUTTON_PIN) == GPIO_PIN_SET)
    {
      if (xSemaphoreTake(pedMutex, portMAX_DELAY) == pdTRUE)
      {
        ns_ped_request = 1; // Store request
        xSemaphoreGive(pedMutex);
      }
    }

    if(HAL_GPIO_ReadPin(GPIOA, EW_BUTTON_PIN) == GPIO_PIN_SET)
    {
      if (xSemaphoreTake(pedMutex, portMAX_DELAY) == pdTRUE)
      {
        ew_ped_request = 1; // Store request
        xSemaphoreGive(pedMutex);
      }
    }
    osDelay(50);
  }
}
```

BLINKTASK

```c
void BlinkTask(void const * argument)
{
  for(;;)
  {
    /* North-South Blue LED Logic */
    // Only blink if there is a request AND NS is currently Green or Yellow
    if (ns_ped_request == 1 && (currentState == STATE_NS_GREEN || currentState == STATE_NS_YELLOW))
    {
      HAL_GPIO_TogglePin(NE_BLUE_PORT, NE_BLUE_PIN);
    }
    else
    {
      HAL_GPIO_WritePin(NE_BLUE_PORT, NE_BLUE_PIN, GPIO_PIN_RESET);
    }

    /* East-West Blue LED Logic */
    // Only blink if there is a request AND EW is currently Green or Yellow
    if (ew_ped_request == 1 && (currentState == STATE_EW_GREEN || currentState == STATE_EW_YELLOW))
    {
      HAL_GPIO_TogglePin(SW_BLUE_PORT, SW_BLUE_PIN);
    }
    else
    {
      HAL_GPIO_WritePin(SW_BLUE_PORT, SW_BLUE_PIN, GPIO_PIN_RESET);
    }

    osDelay(500); // 500ms toggle = 1 second cycle
  }
}
```

- Controls blue indicator LEDs
- Shows active pedestrian requests
- Blinks LEDs for visual feedback
- Runs with lower priority

MUTEX IMPLEMENTATION

- Shared variables:

ns_ped_request, ew_ped_request

- Accessed by multiple tasks
- Risk of race conditions
- Mutex ensures exclusive access
- Prevents data conflicts

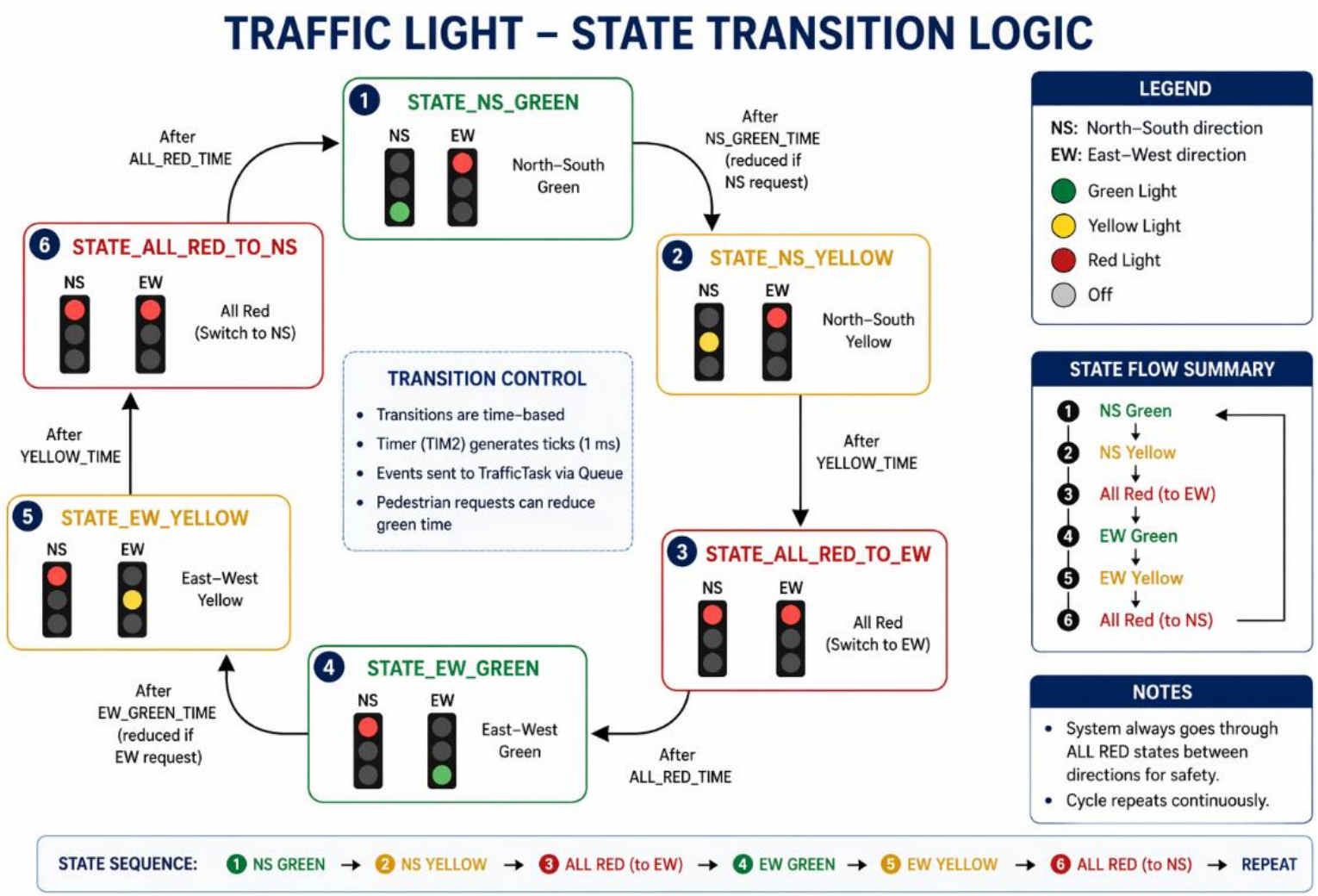
```
/* USER CODE BEGIN RTOS_MUTEX */
pedMutex = xSemaphoreCreateMutex();
/* USER CODE END RTOS_MUTEX */
```

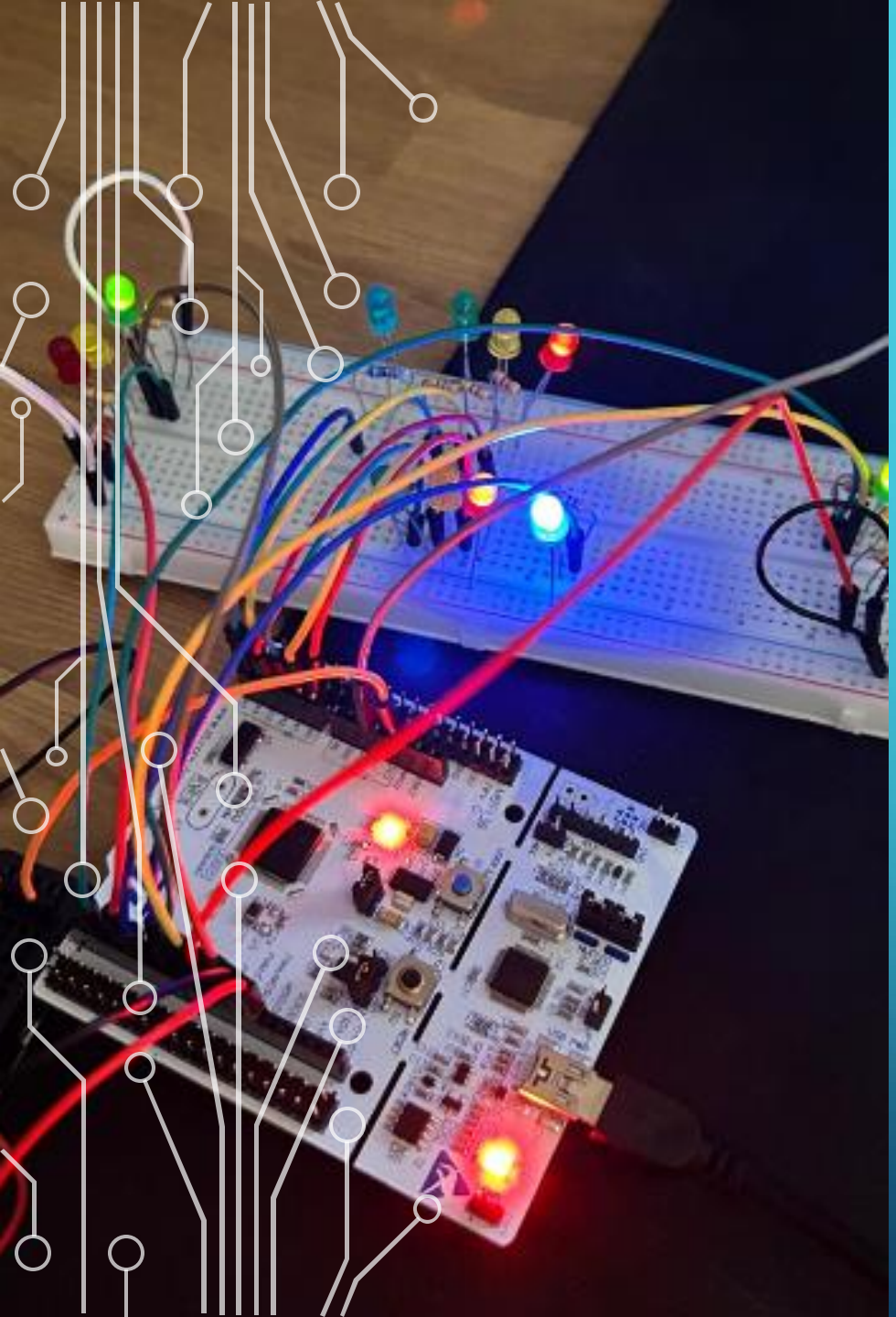
```
void PedestrianTask(void const * argument)
{
  for(;;)
  {
    if(HAL_GPIO_ReadPin(GPIOA, NS_BUTTON_PIN) == GPIO_PIN_SET)
    {
      if (xSemaphoreTake(pedMutex, portMAX_DELAY) == pdTRUE)
      {
        ns_ped_request = 1; // Store request
        xSemaphoreGive(pedMutex);
      }
    }

    if(HAL_GPIO_ReadPin(GPIOA, EW_BUTTON_PIN) == GPIO_PIN_SET)
    {
      if (xSemaphoreTake(pedMutex, portMAX_DELAY) == pdTRUE)
      {
        ew_ped_request = 1; // Store request
        xSemaphoreGive(pedMutex);
      }
    }
    osDelay(50);
  }
}
```

STATE TRANSITION LOGIC

- If pedestrian request exists, green time can shorten to minimum
- Then system transitions to yellow
- After yellow, all-red interval is applied
- Then the opposite direction receives green





RESULTS

- Traffic light sequence works correctly
- Pedestrian requests are detected
- Queue communication works successfully
- Mutex protects shared request variables
- Real-time behavior achieved

CONCLUSION

- STM32 and FreeRTOS were successfully used
- Tasks improved system organization
- Queue enabled event-based communication
- Mutex protected shared variables
- The system simulates a real traffic controller